2141

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра автоматизированных систем управления

Домашнев П.А., Журавлева М.Г.

МЕТОДЫ СОРТИРОВКИ И ПОИСКА В ИНФОРМАЦИОННЫХ МАССИВАХ

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к лабораторным работам
по курсу
«ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ И СТРУКТУРЫ ДАННЫХ»

Липецк
Липецкий государственный технический университет
2014

УДК 004.021

Д-66

Рецензент: кандидат технических наук, В.А. Алексеев

Домашнев, П. А., Журавлева, М.Г.

Д66 Методы сортировки и поиска в информационных массивах [Текст] / Методические указания к лабораторным работам по курсу «Технологии программирования и структуры данных» / П. А. Домашнев, М.Г. Журавлева. – Липецк: ЛГТУ, 2014. – 32 с.

Предназначены для студентов направлений подготовки бакалавров 230100.62 «Информатика и вычислительная техника», 231000.62 «Программная инженерия», 010500.62 «Математическое обеспечение и администрирование информационных систем» и специальности 230401.51 «Информационные системы (по отраслям)».

Приведена краткая теория алгоритмов сортировки и поиска в информационных массивах данных. Содержит задания к двум лабораторным работам.

Табл. 2. Библиогр.: 3 назв.

Теоретические сведения

Сортировка данных

Многие задачи, связанные с обработкой и поиском информации, решаются быстрее, проще и эффективнее, если данные хранятся в памяти ЭВМ в определенном порядке. В ряде случаев польза от упорядоченности данных очевидна и не требует специальных доказательств. Сортировкой или упорядочиванием списка элементов называется расположение этих элементов по возрастанию или убыванию, согласно определенному линейному отношению порядка (для определенности, нижеприведенные алгоритмы предполагают сортировку по возрастанию).

Элементы списка (массива) обычно называют записями. Запись может состоять из ряда информационных полей. Каждая запись характеризуется ключом, по которому осуществляется упорядочивание. Ключом может быть содержимое одного поля записи (ключевого поля) или совокупности определенных полей. В этом случае ключ называется *составным.* Запись также может состоять и из единственного поля, которое в данном случае и будет являться ключевым. В результате упорядочивания записи располагаются по возрастанию или убыванию значений ключей. В общем случае можно определить несколько уровней ключей, при этом старший ключ называется ключом *первого ранга,* а младшие ключи соответственно ключами *второго, третьего рангов* и т.д. Сортировка в этом случае выполняется поэтапно. Вначале записи массива сортируют по ключу первого ранга. Затем записи, имеющие одинаковые значения ключа первого ранга, сортируются по ключу второго ранга и т.д.

В ключевом поле могут храниться числовые или символьные данные. В зависимости от характера ключей записи сортируются либо численным способом, либо алфавитно-цифровым. При численной сортировке записи упорядочиваются в восходящем или нисходящем порядке в соответствии с числовыми значениями ключа. Если в поле ключа хранятся символьные данные, то при сортировке сравниваются строки символов. В результате сортировки устанав-

ливается лексикографический порядок следования записей массива. При сравнении символов сопоставляются двоичные коды их машинного представления. Большим считается символ, имеющий больший код.

В зависимости от состава технических средств, используемых в процессе сортировки, различают *внутреннюю* и *внешнюю* сортировки. Сортировка внутренняя, если весь упорядочиваемый массив целиком помещается в оперативной памяти и находится там в течение всего процесса сортировки. Внешняя сортировка производится в массивах данных, объем которых превышает свободный объем оперативной памяти. В этом случае исходный и отсортированный массивы располагаются на устройствах внешней памяти. В процессе сортировки часть исходного массива считывается в оперативную память, где упорядочивается одним из методов внутренней сортировки, а затем переписывается на внешнее запоминающее устройство. Этот процесс повторяется несколько раз. Полученные упорядоченные последовательности записей затем объединяются. Операция объединения упорядоченных последовательностей данных, размещенных на внешнем запоминающем устройстве, называется *слиянием.* Таким образом, внешняя сортировка включает два этапа обработки: внутреннюю сортировку и слияние.

В общем случае элементы последовательности размещены в любом произвольном порядке. В зависимости от степени упорядоченности исходной последовательности для ее сортировки потребуется то или иное число сравнений и перестановок. При оценке метода сортировки можно довольно просто найти максимальное и минимальное числа сравнений и перестановок. Критериями оценки различных методов обычно являются среднее число операций сравнения, выполняемых в процессе сортировки, и среднее число перестановок или обменов элементов. Эффективность сортировки определяется как частное от деления среднего числа сравнений на число элементов массива.

Процесс сортировки, проводимой любым методом, состоит из нескольких циклов. В каждом цикле осуществляется просмотр всей последовательности и

производятся определенные операции с ее элементами. Один цикл обработки называется *проходом.*

В зависимости от используемого метода сортировки упорядоченная последовательность размещается на том же участке памяти, что и исходная, или требует для своего формирования свободного участка памяти. В первом случае метод минимален по памяти.

В процессе сортировки записи могут физически перемещаться в памяти так, что запись с меньшим ключом окажется расположенной перед записью с большим ключом. Однако не всегда физическая перестановка является обязательной. В ряде случаев достаточно создать вспомогательную таблицу, которая некоторым образом обеспечивает доступ к записям в соответствии с порядком их ключей. Так, например, можно использовать вектор указателей, каждый элемент которого содержит индекс или адрес записи. Порядок следования элементов вектора определяет упорядоченную последовательность элементов основного массива.

Методы внутренней сортировки

Метод выбора

При сортировке этим методом упорядоченная последовательность записей создается на том же участке памяти, что и исходная последовательность. В течение первого прохода осуществляется поиск наименьшего элемента. После того как этот элемент найден, его меняют местами с первым элементом исходной последовательности, в результате чего наименьший элемент занимает первую позицию в формируемой упорядоченной последовательности. Затем осуществляется поиск следующего наименьшего элемента среди оставшихся. Найденный элемент меняется местами со вторым элементом исходной последовательности. После второго прохода окажется сформированной последовательность из двух элементов, первый из которых меньше второго. Поиск элементов со следующими наименьшими значениями ключа и помещение их в со-

ответствующие позиции исходной последовательности продолжается до тех пор, пока все элементы не будут отсортированы в восходящем порядке.

При сортировке рассмотренным методом число сравнений не зависит от степени упорядоченности исходной последовательности.

Среднее число сравнений пропорционально $0{,}5N^2$, среднее число перестановок пропорционально $0{,}5N$, где N – число элементов.

Алгоритм метода выбора приведен ниже:

```
list - сортируемый список элементов
N - число элементов в списке
for i=1 to N do
     minPosition = i
     for j=i+1 to N do
          if(list[minPosition] > list[j]) then
               minPosition = j
          end if
     end for
     tmp = list[minPosition]
     list[minPosition] = list[i]
     list[i] = tmp
end for
```

Метод обмена

При сортировке этим методом упорядоченная последовательность создается на том же месте в памяти, где располагалась исходная последовательность. В процессе сортировки производится попарное сравнение соседних элементов. Если порядок между сравниваемыми элементами нарушен, они меняются местами. Каждый проход начинается с начала списка. Вначале сравниваются первый и второй элементы, затем второй и третий, потом третий и четвертый и так далее; элементы с неправильным порядком в паре переставляются. Если на первом проходе наибольший элемент списка не занимает его последнюю позицию, он будет меняться местами со всеми последующими элементами до тех пор, пока не дойдет до конца списка. Таким образом, в процессе первого про-

хода максимальный элемент попадает на последнее место и, следовательно, в последующих сравнениях не участвует. Остальные элементы "всплывают" на одну позицию вверх (поэтому метод часто называют сортировкой "пузырьком"). На каждом следующем проходе рассматривается последовательность с 1-го по (N-1)-й, (N-2)-й и т.д. элемент, все последующие в сравнении не участвуют, так как, начиная с этой позиции, последовательность уже упорядочена.

После каждого прохода может быть сделана проверка, были ли совершены перестановки в течение данного прохода. Если перестановок не было, то это означает, что последовательность упорядочена и дальнейших проходов не требуется. Максимальное число сравнений составляет приблизительно $0{,}5N^2$, среднее число сравнений пропорционально $0{,}25N^2$, среднее число обменов пропорционально $0{,}25N^2$.

Алгоритм метода обмена приведен ниже:

```
list – сортируемый список элементов
N – число элементов в списке
for i=N to 1 do
     noSwap = 1
     for j=1 to i-1 do
          if(list[j] > list[j+1]) then
               tmp = list[j]
               list[j] = list[j+1]
               list[j+1] = tmp
               noSwap = 0
          end if
     end for
     if(noSwap = 1) then
          break
     end if
end for
```

Метод вставок

Основная идея метода вставок состоит в том, что при добавлении нового элемента в уже отсортированную часть списка, его стоит сразу вставлять в

нужное место вместо того, чтобы вставлять его в произвольное место, а затем заново сортировать весь список.

Первый элемент A_1 исходной последовательности образует уже отсортированную часть списка. Элемент A_2 сравнивается с A_1. Если в результате сравнения оказалось, что $A_2 < A_1$, то элемент A_1 передвигается на одну позицию, а элемент A_2 занимает его прежнее место. Теперь отсортированный список включает два элемента A_1 и A_2, образующих последовательность, упорядоченную по возрастанию значений ключа.

В течение каждого i-го прохода процесса сортировки элемент A_i сравнивается поочередно со всеми элементами, уже составляющими отсортированную последовательность (элементы на позициях с 1 по i-1 списка A), начиная с последнего элемента отсортированной последовательности. Если добавляемый элемент A_i меньше элемента A_j, то элемент A_j перемещается на одну позицию вправо. При обнаружении A_j, меньшего A_i, элемент A_i занимает свободное место j+1 в отсортированной последовательности.

Среднее число сравнений пропорционально $0{,}25N^2$. Минимальное число перестановок равно нулю, и будет это в том случае, когда исходная последовательность уже упорядочена. Среднее число перестановок пропорционально $0{,}25N^2$.

Алгоритм сортировки вставками приведен ниже:

```
list – сортируемый список элементов
N – число элементов в списке
for i=2 to N do
     newElement = list[i]
     location = i-1
     while(location >= 1 and list[location] > newElement) do
          list[location+1] = list[location]
          location = location - 1
     end while
     list[location+1] = newElement
end for
```

Метод подсчета

При использовании этого метода сортировки упорядоченная последовательность B создается на свободном участке памяти. Метод основан на том, что (K + 1)-й элемент упорядоченной последовательности превышает ровно K элементов и, следовательно, занимает (K + 1)-ю позицию. В процессе сортировки на каждом i-м проходе i-й элемент исходной последовательности A попарно сравнивается со всеми остальными элементами. Если в результате сравнения установлено, что $A_i > A_j$, то значение счетчика K увеличивается на единицу. Первоначально счетчик K инициализируется нулем. По окончании прохода счетчик K показывает число элементов, меньших чем A_i. Соответственно элемент A_i в последовательности B занимает позицию K+1.

Для данного метода максимальное, минимальное и среднее число сравнений равно N^2.

Рассмотренный алгоритм сортировки методом подсчета можно использовать лишь в тех случаях, когда в исходной последовательности отсутствуют одинаковые элементы, иными словами, когда в упорядочиваемом массиве нет записей с одинаковыми значениями ключей. Для сортировки массивов, имеющих записи с одинаковыми значениями ключей, алгоритм нужно модифицировать.

Алгоритм метода подсчета приведен ниже:

```
list – сортируемый список элементов
N – число элементов в списке
newList – отсортированный список элементов
for i=1 to N do
     k = 0
     for j=1 to N do
          if(list[i] > list[j]) then
               k = k + 1
          end if
     end for
     newList[k+1] = list[i]
end for
```

Метод Шелла

Этот широко используемый метод, предложенный Д.Л. Шеллом в 1959 г., минимален по памяти и обеспечивает высокую скорость сортировки. Метод предполагает разбиение исходной последовательности на группы элементов и сортировку элементов каждой группы методом вставок. Элементы, принадлежащие одной группе, находятся на определенном расстоянии друг от друга. При необходимости перестановки элементы перемещаются скачком на это расстояние, а не на одну позицию, как в методе вставок.

Для проведения сортировки описываемым методом вначале последовательность из N элементов делится на N/2 или на (N - 1) /2 групп, если N нечетно. Каждая группа содержит по два элемента. Если число элементов нечетно, то одна часть будет содержать три элемента. Элементы, принадлежащие одной группе, отстоят друг от друга на N/2 позиций. Это расстояние называют шагом. В течение первого прохода осуществляется упорядочивание элементов каждой группы методом вставок.

Для осуществления каждого следующего прохода Шелл предложил устанавливать шаг, равный половине предыдущего шага (у дробных чисел берется целая часть). Для последнего прохода устанавливается шаг, равный 1, и упорядочивается единственная группа. В результате попарных сравнений и обменов исходная последовательность оказывается полностью упорядоченной.

Для сортировки последовательности из N элементов требуется около $\log_2 N$ проходов.

Число сравнений, необходимое для сортировки методом Шелла, существенно зависит от шага. До сих пор продолжает обсуждаться вопрос о том, как следует выбирать последовательность шагов. Самим Шеллом была предложена последовательность *N/2, N/4, N/8* и т.д.

До настоящего времени не определена общая формула для расчета среднего числа сравнений. Приближенную оценку числа сравнений производят по формуле $N \log_2 N$.

Алгоритм метода Шелла приведен ниже:

```
list - сортируемый список элементов
N – число элементов в списке
step = N/2
for step = N/2 to 1 do
     ElemCount = N/step
     GroupCount = N/ElemCount
     for i=1 to GroupCount do
          for j=1 to ElemCount do
               NewElement = list[i+j*step]
               location = i+(j-1)*step
               while(location >= i and
                  NewElement < list[location]) do
                    list[location+step] = list[location]
                    location = location - step
               end while
               list[location+step] = NewElement
          end for
     end for
     step = step/2
end while
```

Методы внешней сортировки

Иногда сортируемый список оказывается настолько велик, что он не помещается целиком в оперативную память компьютера. Хотя описанные выше алгоритмы имеют дело с упорядочиванием ключей, подразумевается, что эти ключи связаны с целыми записями. Во многих случаях длина записи значительно превышает длину ключа. Иногда длина записи очень велика, и перестановка двух записей занимает столько времени, что анализ эффективности алгоритма должен учитывать как число сравнений, так и число обменов.

Иногда допустимо объявить массив, размер которого достаточен для размещения всех требуемых данных, хотя размеры этого массива и значительно превышают доступный компьютеру объем памяти. Тогда операционная система пользуется виртуальной памятью, и следует учитывать эффективность ее ис-

пользования. Однако даже и в этом случае объем обменов между оперативной памятью и дисками может быть значительным. Нередко использование рассмотренных методов сортировки для больших массивов данных является неэффективным. В этом случае применяют методы внешней сортировки, которые предусматривают разбиение исходного массива данных на несколько частей и проведение сортировки каждой из частей по отдельности.

Выделяют следующие этапы внешней сортировки:

1 этап. Разбиение на последовательности допустимой длины, которые по отдельности сортируются по какому-либо алгоритму внутренней сортировки. Отсортированные участки записываются на внешнее запоминающее устройство (файл, магнитная лента и т.д.).

2 этап. Слияние - данные с внешнего запоминающего устройства объединяются в единый массив с учетом присутствующей упорядоченности.

Сбалансированное *n*-ленточное слияние

Для n-ленточного слияния потребуется *n* магнитных лент и *n* лентопротяжных устройств (которые можно заменить *n* файлами на устройстве внешней памяти). Исходная неупорядоченная последовательность размещается на первой магнитной ленте. Затем она разносится на *n* магнитных лент по следующему правилу: первая запись – на первую из *n* лент, вторая – на вторую, (*n*+1)-ая – снова на первую из *n* лент.

Количество лент *n* при этом вычисляется как $n = \sqrt{N}$, где *N* – общее количество элементов в исходном массиве.

Сбалансированное n-ленточное слияние осуществляется в два этапа. На первом этапе из записей, хранящихся на каждой магнитной ленте, формируются упорядоченные цепочки. Так как все цепочки имеют одинаковую длину, слияние называется сбалансированным. Упорядочение цепочки происходит в оперативной памяти одним из методов внутренней сортировки. При этом требуется в $\sqrt{N}$ раз меньше оперативной памяти, чем было бы необходимо для применения того же метода внутренней сортировки для упорядочивания всего

исходного массива, т.к. количество элементов в каждой цепочке не превышает $\sqrt{N}$.

После этого начинается второй этап сортировки – слияние. Процесс слияния осуществляется в несколько этапов. На каждом этапе просматриваются первые элементы каждой из непустых лент. Из них выбирается (по порядку, за *n* шагов) минимальный элемент и записывается в результирующую ленту на текущую позицию. Таким образом, одновременно в оперативной памяти будет присутствовать не более $\sqrt{N}$ элементов. После этого текущая позиция результирующей ленты сдвигается вправо. После каждого шага длина упорядоченной последовательности увеличивается на один элемент. Если из какой-либо ленты «вытащен» последний элемент, такая лента в дальнейшем процессе слияния не участвует. Процесс завершается, когда останется только одна непустая лента. Все оставшиеся в ней элементы переносятся в результирующую последовательность.

Поиск данных

Основные принципы информационного поиска

Основной задачей информационного поиска является решение вопросов по соответствию данных, содержащихся в записи массива установленным критериям поиска. Результатом любого поиска является список всех записей удовлетворяющих критерию поиска или сообщение о том, что таких записей в информационном массиве нет.

В зависимости от запроса, аргумент поиска может иметь различную форму и степень сложности. В простом случае, когда необходимо найти запись, характеризующую объект, обладающий определенным признаком, аргументом поиска будет являться этот признак. Данный вид поиска называют *одноаспектным*. Если аргумент поиска содержит перечень признаков, то такой поиск называется *многоаспектным*. Аргумент может представлять из себя формулу булевой алгебры или теории множеств, содержать перечень признаков и логи-

ческих операций над ними либо операции теории множеств (объединение, пересечение, разность).

Существуют следующие виды информационного поиска:

1. Поиск по совпадению. Аргумент поиска содержит один или несколько признаков и их значения. В процессе поиска из массива выделяются записи, значения соответствующих признаков которых совпадают со значениями, приведенными в аргументе поиска.
2. Поиск по интервалу. В отличие от поиска по совпадению, аргумент поиска содержит границы изменения значений признаков. Попадание в эти границы сигнализирует о выполнении условий запроса.
3. Поиск по выражению. Признаки в аргументе поиска объединены в выражение с помощью логических или теоретико-множественных операций. Истинность этого выражения для значения признаков записи говорит об удовлетворении записи запросу.

Методы поиска

Одноаспектный поиск

Различают последовательный и ускоренный поиск. Применение последовательного поиска не требует проведения никаких предварительных преобразований информационного массива. Методы ускоренного поиска требуют проведения предварительного упорядочивания информационного массива (двоичный и блочный поиск) или предварительного построения вспомогательных информационных структур (методы поиска с использованием общего и единого справочника).

Метод последовательного поиска

Данный метод является простейшим алгоритмом поиска в информационном массиве и не накладывает никаких ограничений на распределение элементов массива.

Алгоритм заключается в последовательном считывании записей массива, по каждой из которых принимается решение о выдаче в качестве результата в соответствии с аргументом поиска. Перед каждой процедурой считывания записи осуществляется проверка достижения конца массива. Среднее число операций сравнения – $0{,}5N$ (N – объем массива).

В связи с малой эффективностью по сравнению с другими алгоритмами линейный поиск обычно используют, только если массив содержит очень мало элементов, тем не менее, линейный поиск не требует дополнительной памяти или предварительной обработки массива, так что может работать в потоковом режиме при непосредственном получении данных из любого источника.

Двоичный поиск

Двоичный (бинарный) поиск является классическим алгоритмом поиска элемента в отсортированном массиве, использующим дробление массива на половины.

Идея метода заключается в следующем: аргумент поиска сравнивается со средним элементом массива; если значение аргумента поиска меньше среднего элемента, то исходной считается последовательность от первого элемента до элемента с номером N/2, иначе, если значение аргумента поиска больше текущего элемента – последовательность от элемента с номером N/2 до N; затем для новой последовательности элементов данный процесс повторяется. В целом, осуществляется последовательное деление пополам интервала поиска до тех пор, пока не будет найден искомый элемент или длина последовательности не станет равной единице. Таким образом, на каждой итерации массив делится пополам и отбрасывается часть массива, заведомо не содержащая требуемую запись.

Максимально необходимое количество сравнений $\log_2 N$, минимальное количество сравнений - 1. Среднее количество сравнений – $\log_2(N-1)$.

Алгоритм двоичного поиска приведен ниже:

```
list – отсортированный список элементов
N – число элементов в списке
```

```
arg – аргумент поиска
left = 1
right = N
resIndex = -1;
while (right <> left) do
     middle = (right+left)/2
     if (arg = list[middle]) then
          resIndex = middle
          break;
     end if
     if (arg > list[middle]) then
          left = middle
     else
          right = middle
     end if
end while
if (list[right] = arg) then
     resIndex = right;
end if
if (resIndex = -1) then
     print ‘Элемент не найден’
else
     print ‘Элемент найден’
end if
```

Блочный поиск

Исходный массив делится на блоки. Поиск осуществляется в два этапа. Во время первого этапа аргумент поиска сравнивается с последними элементами каждого из блоков. Если не будет получено значение, превосходящее аргумент поиска, результат поиска – отрицательный, иначе выполняется второй этап, в ходе которого осуществляется последовательный поиск в найденном блоке в направлении от последней к первой записи в блоке. Практика показывает, что $\sqrt{N}$ является лучшим количеством блоков для N записей. В среднем требуется $\sqrt{N}$ сравнений, в худшем случае – $2\sqrt{N}$ сравнений.

При необходимости каждый из блоков может быть дополнительно поделен на подблоки, но использование более двух уровней делений нецелесообразно.

Следует отметить, что деление на блоки не выполняется буквально. Все элементы хранятся в едином упорядоченном информационном массиве.

Алгоритм блочного поиска приведен ниже:

```
list – отсортированный список элементов
N – число элементов в списке
arg – аргумент поиска
blockCount = sqrt(N)
resIndex = -1;
for i = blockCount to N step = blockCount do
     if (list[i] > arg) then
          for j=i to i-blockCount step = -1 do
               if (list[j] = arg) then
                    resIndex = j;
                    break;
               end if;
               if (list[j] < arg) then
                    break;
               end if;

          end for
          break
     end if
end for
if (resIndex = -1) then
     print ‘Элемент не найден’
else
     print ‘Элемент найден’
end if
```

Ускоренный поиск, основанный на использовании справочников

Часто записи информационного массива имеют большое количество информационных полей и значительный размер. В этом случае операции обмена информационных записей, выполняемые в ходе предварительной сортировки информационного массива, требуют значительных временных ресурсов, что приводит к значительному увеличению времени сортировки. Кроме того, сама процедура двоичного и блочного поиска является затратной по памяти, так как требует загрузки в оперативную память всего информационного массива.

Для оптимизации этих процессов используют дополнительные служебные структуры, которые называются справочниками и формируются в процессе предварительной обработки информационного массива. Различают общий и единый справочники.

Общий справочник создается на основе неупорядоченного массива, содержащего записи фиксированной или переменной длины. В нем для каждой записи основного массива создается одна справочная запись, называемая статьей. Статья содержит значения ключевого поля и указатель, определяющий адрес записи в информационном массиве. Статьи в справочнике упорядочиваются по значениям ключа. При этом сам исходный информационный массив не подвергается никаким преобразованиям.

Непосредственно поиск ведется по построенному справочнику, при этом для поиска можно использовать двоичный и блочный поиск. После успешного поиска по справочнику по полученному указателю происходит выбор требуемой записи из информационного массива. При пополнении информационного массива происходит пополнение справочника, при условии сохранения упорядоченности.

Использование общего справочника позволяет исключить физическое переупорядочивание информационного массива, тем самым ускоряя процесс предварительной подготовки к применению методов ускоренного поиска.

В случае использования единого справочника информационный массив предварительно упорядочивается по значению ключа и разбивается на блоки. Каждый блок содержит заданное количество записей. Для каждого блока в еди-

ном справочнике создается статья, содержащая поле ключа и поле указателя. Поле указателя содержит адрес (индекс) первой записи в блоке. Поле ключа – значение ключа последней записи этого блока. Поиск осуществляется в два этапа. Сначала по единому справочнику находится статья, значение поля ключа которой больше, чем аргумент поиска. Затем проводится последовательный поиск в информационном массиве, начиная со значения поля указателя данной статьи – адреса первой записи найденного блока.

При большом объеме информационного массива единый справочник может оказаться настолько велик, что потребуется также разбить его на блоки, создав справочник следующего, второго уровня. Создание справочников выше третьего уровня не является целесообразным. Оптимальный размер блока одноуровнего справочника равен квадратному корню из количества записей массива.

Стратегия реализации одноаспектного поиска по совпадению

Все рассмотренные методы ускоренного поиска ориентированы на обнаружение одной единственной записи в информационном массиве (определении индекса записи удовлетворяющей критерию поиска). Тогда как при практическом использовании задачей поиска по совпадению, является локализация всех информационных записей удовлетворяющих критерию поиска. Для достижения этой цели требуется в случае положительного окончания процедуры ускоренного поиска, выявить все соседние информационные записи с аналогичными значениями ключа. Т.е. последовательно проверить записи, находящиеся слева и справа от найденной записи, сверив значения их ключей с аргументом поиска. Таким образом, результатом поиска по совпадению будет набор индексов информационного массива, заданный интервалом [A;B], в котором содержатся записи, удовлетворяющие критерию поиска.

Соответствующий алгоритм для случая применения двоичного поиска приведен ниже:

```
list - отсортированный список элементов
```

```
N – число элементов в списке
arg – аргумент поиска
left = 1
right = N
resIndex = -1;
while (right <> left) do
     middle = (right+left)/2
     if (arg = list[middle]) then
          resIndex = middle
          break;
     end if
     if (arg > list[middle]) then
          left = middle
     else
          right = middle
     end if
end while
if (list[right] = arg) then
     resIndex = right
end if
if (resIndex = -1) then
     print ‘Элемент не найден’
else
     leftBorder = resIndex
     rightBorder = resIndex
     while (leftBorder > 0  and list[leftBorder] = arg) do
          leftBorder = leftBorder - 1
     end while
     leftBorder = leftBorder + 1
     while (rightBorder < N and list[rightBorder] = arg) do
          rightBorder = rightBorder + 1
     end while
     rightBorder = rightBorder - 1
     print ‘Найдены элементы:’
     for i = leftBorder to rightBorder do
```

```
          print list[i]
     end for
end if
```

Стратегия реализации одноаспектного поиска по интервалу

Задача процедуры одноаспектного поиска по интервалу заключается в выявлении множества информационных записей, значения ключевых полей которых находятся в заданных пределах. В случае применения метода последовательного поиска для локализации всех таких записей требуется просмотреть весь информационный массив. В случае же использования ускоренного поиска такой необходимости нет, так как исходный информационный массив тем или иным способом упорядочивается. Поэтому достаточно обнаружить в упорядоченном информационном массиве записи, соответствующие минимальному и максимальному значению ключа заданным в аргументе поиска, при этом все искомые информационные записи будут находиться между ними.

Таким образом, для выполнения поиска по интервалу необходимо выполнить два последовательных ускоренных поиска по значению: для левой границы интервала и для правой границы. Результатом каждого поиска будет либо индекс искомой записи (в случае успешного поиска), либо индекс позиции, на которой могла бы быть искомая запись (в случае отсутствия таковой в информационном массиве). Это требует незначительной модификации процедуры поиска по значению. Пример реализации этого алгоритма для случая использования двоичного поиска приведен ниже.

```
list – отсортированный список элементов
N – число элементов в списке
arg – аргумент поиска
typeOfBorder – тип границы, которая ищется: 1 – левая, 2 - правая
left = 1
right = N
resIndex = -1;
while (right <> left) do
     middle = (right+left)/2
```

```
     if (arg = list[middle]) then
          resIndex = middle
          break;
     end if
     if (arg > list[middle]) then
          left = middle
     else
          right = middle
     end if
end while
if (list[right] = arg) then
     resIndex = right
end if
if (resIndex = -1) then
     if (typeOfBorder = 1) then
          print right
     else
          print right - 1
     end if
else
     if (typeOfBorder = 1) then
          while (list[resIndex] = arg) do
               resIndex = resIndex - 1
          end while
          resIndex = resIndex + 1
     else
          while (list[resIndex] = arg) do
               resIndex = resIndex + 1
          end while
          resIndex = resIndex - 1
     end if
     print resIndex
end if
```

Многоаспектный поиск

Метод последовательного поиска

Также как и в случае с одноаспектным поиском данный метод является простейшим алгоритмом многоаспектного поиска в информационном массиве и не накладывает никаких ограничений на распределение элементов массива.

Записи информационного массива считываются последовательно, и по каждой из них принимается решение о соответствии записи критериям (аргументам) поиска. Перед каждой процедурой считывания записи осуществляется проверка достижения конца массива. Среднее число операций сравнения – N/2 (N – объем массива).

Ускоренный многоаспектный поиск с использованием инверсных массивов

Для проведения ускоренного многоаспектного поиска требуется предварительная обработка массива информационных данных:

1. Определяется список полей по которым может быть осуществлен запрос на многоаспектный поиск. Пусть имеется m таких полей.
2. Создается m копий информационного массива, каждая из которых упорядочивается по одному из m выбранных полей. Эти копии называются инверсными массивами.

Непосредственно многоаспектный поиск, включающий m аргументов, выполняется в несколько этапов:

1. В каждой из копий информационного массива проводится одноаспектный поиск по соответствующему аргументу поиска (полю, по которому упорядочен массив). Для поиска может быть использован двоичный или блочный метод. Результатом каждого такого поиска является множество адресов (индексов) элементов исходного информационного массива, удовлетворяющих критерию соответствующего одноаспектного поиска.

2. Над всеми полученными множествами проводится операция теоретико-множественного пересечения. Полученное в результате этого множество адресов (индексов) элементов исходного информационного массива и является результатом многоаспектного поиска.

Более эффективно вместо создания отдельных копий информационного массива создавать общие справочники, и в дальнейшем применять ускоренный поиск с использованием этих общих справочников.

Задания к лабораторному практикуму

Лабораторная работа №1. Программирование базовых алгоритмов сортировки

Цель работы

Приобретение навыков реализации базовых алгоритмов сортировки массивов данных.

Задание кафедры

Осуществить программную реализацию сортировки информации заданного вида сбалансированным N-ленточным слиянием (в оперативной памяти), используя выбранные из табл. 1, в соответствии с вариантом, алгоритм внутренней сортировки и формат исходных данных.

Контрольные вопросы

Содержание отчета

Отчет оформляется согласно общим требованиям, предъявляемым к оформлению лабораторных работ и должен содержать:

1. Титульный лист.
2. Задание кафедры, соответствующее варианту, номер варианта.
3. Цель работы.
4. Краткие теоретические сведения

5. Блок-схему алгоритма разработанной программы
6. Текст программы
7. Контрольный пример
8. Выводы по работе.

Оформление отчета осуществляется в соответствии с СТО-13-2011, при этом в описание экспериментальной установки и методики эксперимента следует включить блок-схему алгоритма разработанной программы и текст программы.

Рекомендуемый порядок выполнения лабораторной работы:

1. Построение алгоритма решения задачи.

2. Написание программы: проверка корректности вводимых пользователем данных (при вводе пользователем некорректных данных следует выводить сообщение об ошибке); написание функций, выполняющих требуемые операции; реализация интерфейса пользователя.

3. Тестирование программы, обработка результатов работы.

Таблица 1

Задания к лабораторной работе № 1

№ п/п	Тип ключевого поля записи	Типы информационных полей записи	Метод внутренней сортировки
1	int	char[], float	Метод выбора
2	int[3]	-	Метод вставок
3	float	float[], char[]	Метод обмена
4	float[3]	-	Метод Шелла
5	char	int[], char	Метод подсчета
6	char[20]	-	Метод выбора
7	int	char, float	Метод вставок
8	int[3]	-	Метод обмена
9	float	int[], char	Метод Шелла
10	float[3]	-	Метод подсчета
11	char	-	Метод выбора
12	char[20]	-	Метод вставок

13	int	char[]	Метод обмена
14	int[3]	-	Метод Шелла
15	float	char[], int	Метод подсчета
16	float[3]	-	Метод выбора
17	char	char[], int	Метод вставок

Продолжение табл. 1

18	char[20]	-	Метод обмена
19	int	char[], float[]	Метод Шелла
20	int[3]	-	Метод подсчета
21	float	char[], int, int	Метод выбора
22	float[3]	-	Метод вставок
23	char	int[5], float	Метод обмена
24	char[20]		Метод Шелла
25	int[3]	char[], int	Метод подсчета

Лабораторная работа №2. Программирование базовых алгоритмов поиска

Цель работы

Приобретение навыков реализации базовых алгоритмов программного поиска в информационных массивах.

Задание кафедры

Написать программу, реализующую один из алгоритмов программного поиска данных в информационном массиве, расположенном в оперативной памяти (по желанию, можно считывать данные из файла), используя выбранные в соответствии с вариантом из таблицы формат ключа, формат других полей записи, вид и метод поиска.

Контрольные вопросы

Содержание отчета

Отчет оформляется согласно общим требованиям, предъявляемым к оформлению лабораторных работ и должен содержать:

1. Титульный лист.
2. Задание кафедры, соответствующее варианту, номер варианта.
3. Цель работы.
4. Краткие теоретические сведения
5. Блок-схему алгоритма разработанной программы
6. Текст программы
7. Контрольный пример
8. Выводы по работе.

Оформление отчета осуществляется в соответствии с СТО-13-2011, при этом в описание экспериментальной установки и методики эксперимента следует включить блок-схему алгоритма разработанной программы и текст программы.

Рекомендуемый порядок выполнения лабораторной работы:

1. Построение алгоритма решения задачи.

2. Написание программы: проверка корректности вводимых пользователем данных (при вводе пользователем некорректных данных следует выводить сообщение об ошибке); написание функций, выполняющих требуемые операции; реализация интерфейса пользователя.

3. Тестирование программы, обработка результатов работы.

Таблица 2

Задания к лабораторной работе № 2

№ п/п	Тип ключевого поля записи	Типы информационных полей записи	Тип информационного поиска	Метод поиска

1	int	char[], float	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Двоичный поиск
2	int[3]	char[], int	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Блочный поиск
3	float	float[], char[]	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием общего справочника (двоичный поиск)

Продолжение табл. 2

4	float[3]	char[], char[]	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием общего справочника (блочный поиск)
5	char	int[], char	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием единого справочника
6	int	char[], float	Многоаспектный поиск по совпадению	Двоичный поиск
7	int[3]	char[], int	Многоаспектный поиск по совпадению	Блочный поиск
8	float	float[], char[]	Многоаспектный поиск по совпадению	Поиск с использованием общего справочника (двоичный поиск)
9	float[3]	char[], char[]	Многоаспектный поиск по совпадению	Поиск с использованием общего справочника (блочный поиск)
10	char	int[], char	Многоаспектный поиск по совпадению	Поиск с использованием единого справочника
11	char[20]	char[], int, float	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Двоичный поиск
12	int[2]	char[], int[]	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Блочный поиск
13	float[3]	int[], char[], float	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием общего справочника (двоичный поиск)

14	float	char[], int	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием общего справочника (блочный поиск)
15	float[3]	int[], int[], char	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием единого справочника
16	char[20]	char[], int, float	Многоаспектный поиск по совпадению	Двоичный поиск
17	int[2]	char[], int[]	Многоаспектный поиск по совпадению	Блочный поиск

Продолжение табл. 2

18	float[3]	int[], char[], float	Многоаспектный поиск по совпадению	Поиск с использованием общего справочника (двоичный поиск)
19	float	char[], int	Многоаспектный поиск по совпадению	Поиск с использованием общего справочника (блочный поиск)
20	float[3]	int[], int[], char	Многоаспектный поиск по совпадению	Поиск с использованием единого справочника
21	int	char[], int, float	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Двоичный поиск
22	int[3]	char[], int[]	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Блочный поиск
23	float	int[], char[], float	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием общего справочника (двоичный поиск)
24	float[3]	char[], int	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием общего справочника (блочный поиск)
25	char	int[], int[], char	Одноаспектный поиск по совпадению и одноаспектный поиск по интервалу	Поиск с использованием единого справочника
26	int	char[], int, float	Многоаспектный поиск по совпадению	Двоичный поиск
27	int[3]	char[], int[]	Многоаспектный поиск по совпадению	Блочный поиск

28	float	int[], char[], float	Многоаспектный поиск по совпадению	Поиск с использованием общего справочника (двоичный поиск)
29	float[3]	char[], int	Многоаспектный поиск по совпадению	Поиск с использованием общего справочника (блочный поиск)
30	char	int[], int[], char	Многоаспектный поиск по совпадению	Поиск с использованием единого справочника

Список литературы

1. Кнут, Д. Э. Искусство программирования, том 3. Сортировка и поиск / Д. Э. Кнут. – М.: "Вильямс", 2012. – 824 с.
2. Макконнелл, Дж. Основы современных алгоритмов / Дж. Макконнелл. – М.: Техносфера, 2004. – 368 с.
3. Ахо, А. В. Структуры данных и алгоритмы / А. В. Ахо, Дж. Хопкрофт, Дж. Д. Ульман. – М.: "Вильямс", 2010. – 400 с.

Павел Алексеевич Домашнев, Журавлева Марина Гарриевна

МЕТОДЫ СОРТИРОВКИ И ПОИСКА В ИНФОРМАЦИОННЫХ МАССИВАХ

Методические указания к лабораторным работам
по курсу
«ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ И СТРУКТУРЫ ДАННЫХ»

Редактор

Подписано в печать . Формат 60x84 1/16. Бумага офсетная.

Ризография. Объем 2,0 л. Тираж 100 экз. Заказ № .

Издательство Липецкого государственного технического университета.

Полиграфическое подразделение Издательства ЛГТУ.

398600, Липецк, ул. Московская, 30.